

CLEANROOMX

Engineering User & Validation Manual

Design • Simulation • HVAC • Networks • BIM/IFC • Verification • Evidence • Controlled Handoff

Document	Engineering User & Validation Manual
Revision	2.0
Stable release	v0.102.1
Engineering baseline	ae80aec3f61f009597a1b01f05d67fb91b545d70 (PR #594 engineering baseline)
Manual date	30 September 2026

Engineering and regulatory boundary

CleanroomX provides engineering screening, simulation, verification, and software/provenance evidence. It does not by itself establish cleanroom certification, CFD validation, commissioning/TAB acceptance, manufacturer approval, or regulatory compliance. Applicable criteria remain explicit project inputs and require qualified review.

Intended audience: cleanroom/HVAC engineers, BIM coordinators, verification engineers, reviewers, QA/validation personnel, technical leads, and controlled-document owners.

This revision reorganizes the manual around operator tasks, review gates, and industrial use rather than repository chronology. Development status is isolated in the final appendix.

Contents

1. Document Control, Scope, and Intended Use

8

1.1 Purpose of this manual

8

1.2 Document status

8

1.3 Status labels used in this manual

9

1.4 Intended audience

9

1.5 Responsibility boundary

9

1.6 Recommended controlled-use pattern

9

2. Product Architecture and Engineering Philosophy

11

2.1 What CleanroomX is

11

2.2 Engineering principles

11

2.3 Capability map

12

2.4 Lifecycle view

12

3. Installation, Readiness, and Controlled Deployment

13

3.1 Supported environment

13

3.2 Installation acceptance procedure

13

3.3 Production deployment guidance

14

3.4 Linux / CI GUI smoke

14

4. Project Lifecycle and File Governance

15

4.1 Project file model

15

4.2 Strict JSON boundary

15

4.3 Additive-field preservation

15

4.4 Guarded project saves

16

4.5 Legacy migration

16

4.6 Recovery autosave 16

5. Desktop GUI - Standard Operating Procedure 17

5.1 Normal operator sequence 17

5.2 Run ownership and Abandon 17

5.3 Result freshness 17

5.4 Review the four panes of evidence 18

5.5 Recommended analysis naming 18

6. Spatial Design Workspace 19

6.1 Canonical 2D/3D model 19

6.2 Supported device/opening types 19

6.3 Editing procedure 19

6.4 3D navigation 20

6.5 Spatial validation 20

7. Spatial to Engineering Synchronization 21

7.1 Synchronization boundary 21

7.2 Synchronization states 21

7.3 Recommended conflict procedure 21

8. BIM / IFC Interoperability 23

8.1 Purpose 23

8.2 Current mapping scope 23

8.3 Geometry fidelity boundary 23

8.4 Initial import SOP 23

8.5 Re-import SOP 24

8.6 IFC resource and source-stability guards 24

9. Room and Project Verification 25

9.1 Room volume	25
9.2 Nominal supply ACH	25
9.3 Particle requirements	25
9.4 Observed pressure and pressure cascade	25
9.5 Aggregate result semantics	25
9.6 Worked demonstration from the packaged project	26
10. Particle Decay and Recovery Qualification	27
10.1 First-order decay model	27
10.2 Recovery time	27
10.3 What the screening model does not include	27
10.4 Measured recovery workflow	27
11. Psychrometrics, Thermal Loads, and HVAC Screening	28
11.1 Airflow balance	28
11.2 Preliminary supply-fan duty	28
11.3 Psychrometric uncertainty	28
11.4 Thermal uncertainty	28
12. Duct, Branch, and Looped Airflow Networks	30
12.1 Duct section model	30
12.2 Duct-path review checklist	30
12.3 Fixed-demand branch network	30
12.4 Looped network model	30
12.5 Numerical method	31
13. Room Pressure / Leakage Network - Current Main	32
13.1 Purpose	32
13.2 Mechanical injection	32

13.3 Power-law path 32

13.4 Orifice path 32

13.5 Near-zero regularization 33

13.6 Pressure targets 33

13.7 Worked pressure-network demonstration 33

14. Fan/System Studies and Network Coupling 34

14.1 Supplied fan-curve operating point 34

14.2 Fan curve review questions 34

14.3 Fan speed studies 34

14.4 Fan-network and fan-loop integrations 34

14.5 Pending PR #595 at manual cutoff 34

15. Variable Friction, Damper Scenarios, and Numerical Integrity 36

15.1 Variable-friction loop workflow 36

15.2 Near-zero flow handling 36

15.3 Damper-resistance scenario studies 36

15.4 Why full precision matters 36

16. Design Requirements, Consistency, Compliance, and Assurance - Current Main 38

16.1 Design requirements 38

16.2 Preliminary air-system design 38

16.3 Design consistency 38

16.4 Pressure design consistency 39

16.5 Compliance rule packs 39

16.6 Design assurance matrix 39

17. ProofGraph Evidence Model - Current Main 40

17.1 Purpose 40

17.2 Core objects	40
17.3 Fail-closed structural rules	40
17.4 Verdict states	41
17.5 Corrective actions	41
17.6 Evidence integrations	41
18. Reporting, Bundles, Snapshots, Batch, and Diagnostics	42
18.1 Portable HTML engineering report	42
18.2 Portable project bundle	42
18.3 Design assurance snapshot	42
18.4 Deterministic project batch execution	42
18.5 Project diagnostics	43
19. Industry Review Gates and Evidence Release	44
19.1 Recommended gate model	44
19.2 Reviewer checklist	44
19.3 Recommended released package	45
20. Security, Data Integrity, and Operational Controls	46
20.1 Security boundary	46
20.2 Project input controls	46
20.3 Hashes and authenticity	46
21. Troubleshooting and Failure Handling	47
21.1 GUI does not start	47
21.2 Project will not open	47
21.3 Save blocked because the file changed externally	47
21.4 Result disappeared after editing input	47
21.5 IFC re-import reports conflict	47

21.6 Network solver does not converge

47

21.7 Fan study reports no intersection

48

21.8 PASS but incomplete

48

22. Validation, Change Control, and Rollback

49

22.1 Stable release evidence

49

22.2 Controlled change procedure

49

22.3 Code rollback

49

22.4 Project rollback

49

23. Command-Line Quick Reference

51

24. Engineering Formula Summary

53

25. External Standards and Reference Map

54

26. Final Release Checklist

55

Software and project identity

55

Model and input review

55

Analysis review

55

Evidence and handoff

55

27. Glossary and Status Semantics

57

28. Current Development Status at Manual Cutoff

58

28.1 Stable v0.102.1

58

28.2 Current-main engineering additions through PR #594

58

28.3 Pending PR #595

59

28.4 Authoritative repository records

59

CHAPTER 1

Document Control, Scope, and Intended Use

1. Document Control, Scope, and Intended Use

1.1 Purpose of this manual

This manual is the primary operating and engineering reference for CleanroomX. It is written for real project use: installation, model setup, engineering analysis, review, evidence production, BIM/IFC coordination, troubleshooting, deployment, and controlled handoff.

The manual is deliberately task-oriented. It separates stable released capability from unreleased capability on main, and it separates software evidence from engineering or regulatory approval. Repository history and implementation detail remain available in the project documentation, but they are not allowed to obscure day-to-day operating procedures.

Manual objective

A competent engineer or reviewer should be able to install CleanroomX, establish a project, execute a supported workflow, identify whether the result is current and complete, package the evidence, and understand the engineering limitations without reading source code.

1.2 Document status

Item	Controlled value
Manual revision	2.0
Manual date	30 September 2026
Stable software release	v0.102.1
Stable release validation	1008 passing tests on Python 3.11, 3.12, and 3.13 at the final v0.102.1 gate
Engineering implementation baseline used for this manual	ae80aec3f61f009597a1b01f05d67fb91b545d70 - PR #594
Project schema	cleanroomx.project, schema version 1
ProofGraph schema	cleanroomx.proofgraph, schema version 1 - current-main capability
Supported Python	3.11 / 3.12 / 3.13
Pending at cutoff	PR #595, CI run #1928 SUCCESS, not merged into the engineering baseline

Later documentation-only commits may exist because this manual is published from the same repository. For engineering reproducibility, the software build used for a project should be recorded independently in the project evidence.

1.3 Status labels used in this manual

Label	Meaning
Stable	Capability is part of the v0.102.1 release closure and release evidence.
Current main	Capability is implemented after v0.102.1 on the verified engineering baseline used for this manual; it is not part of the immutable v0.102.1 release tag.
Pending	Capability exists only in an open pull request at the manual cutoff and shall not be treated as available on main.

1.4 Intended audience

This manual is written for the following roles.

Role	Primary responsibility in CleanroomX
Cleanroom / HVAC design engineer	Define engineering inputs, execute analyses, review assumptions and margins.
Verification engineer	Check model validity, solver status, completeness, uncertainty, and traceability.
BIM coordinator	Import/review IFC semantics, manage GlobalId identity, resolve re-import conflicts.
Technical reviewer	Independently review assumptions, inputs, results, limitations, and report freshness.
QA / validation representative	Control software version, evidence package, test status, approvals, and records.
System owner / IT	Deploy supported Python/Tk environment, file permissions, backup, and access controls.
Project manager / lead	Define acceptance criteria, review gates, deliverables, and responsible approvers.

1.5 Responsibility boundary

CleanroomX calculates and preserves evidence from explicit inputs. It does not decide what a project is legally required to achieve. Cleanroom classification limits, pressure targets, recovery requirements, thermal limits, filter criteria, commissioning acceptance limits, and manufacturer constraints must come from the applicable project requirements and selected standards.

Do not convert a software PASS into regulatory approval

A CleanroomX PASS means the configured CleanroomX check passed under the supplied model and criteria. It is not, by itself, cleanroom certification, GMP acceptance, TAB acceptance, manufacturer approval, CFD validation, or regulatory approval.

1.6 Recommended controlled-use pattern

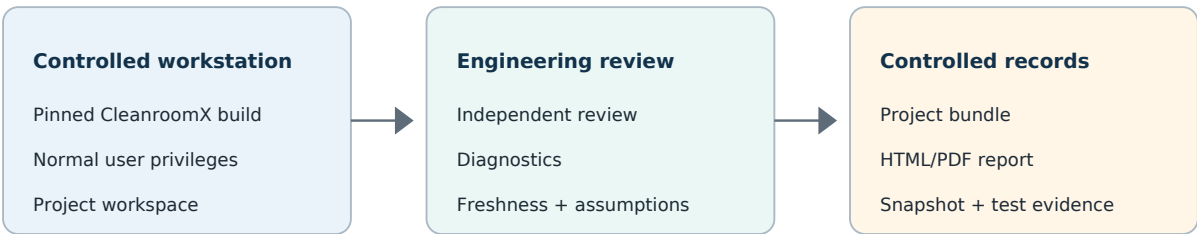


Figure 1-1. Recommended use of CleanroomX inside an engineering or quality-controlled organization.

For controlled projects, separate these responsibilities wherever practical:

- software installation and version control;
- engineering input preparation;
- analysis execution;
- independent technical review;
- QA/validation approval of released evidence;
- document retention and change control.

This separation reduces the risk that a single operator both defines a criterion and approves the resulting evidence without independent review.

CHAPTER 2

Product Architecture and Engineering Philosophy

2. Product Architecture and Engineering Philosophy

2.1 What CleanroomX is

CleanroomX is a local Python engineering platform for cleanroom design, simulation, verification, spatial planning, HVAC analysis, airflow networks, fan/network studies, bounded uncertainty, BIM/IFC interoperability, reporting, and auditable numerical/provenance evidence.

The desktop GUI, command-line tools, and project batch runner use shared backend services. Engineering equations are not reimplemented inside GUI callbacks. This is an important quality property: the same canonical engineering services are reused across interactive, automated, reporting, and assurance workflows.

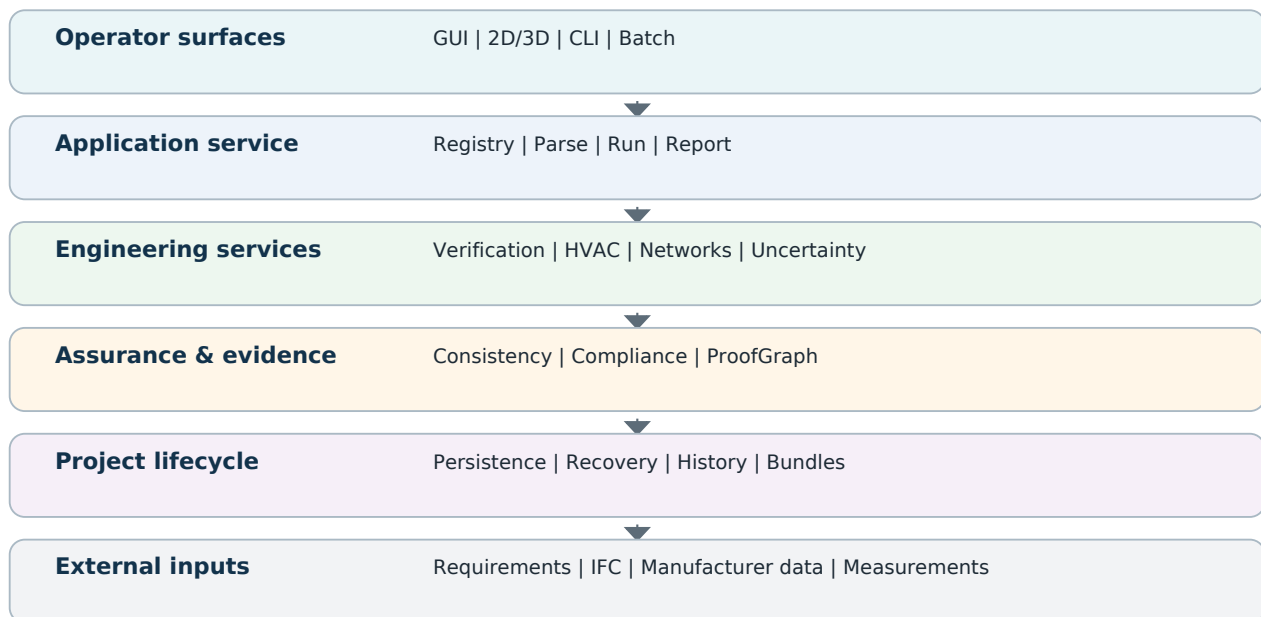


Figure 2-1. CleanroomX architecture - one canonical engineering stack behind multiple operator surfaces.

2.2 Engineering principles

CleanroomX development follows a small set of principles that operators should understand because they affect how results must be interpreted.

- Explicit inputs over hidden assumptions. Project criteria, leakage coefficients, fan curves, efficiencies, uncertainty bounds, and standard-derived limits are entered or imported explicitly.
- Full-precision calculation before presentation rounding. Current numerical hardening keeps canonical floating-point state through calculations and rounds only at presentation boundaries.

- Fail closed where provenance is incomplete or ambiguous. Missing mappings, stale dependencies, invalid JSON, conflicting IFC re-imports, and invalid ProofGraph links do not silently become success.
- Deterministic evidence. Supported workflows preserve canonical input identities, SHA-256 content identities, solver provenance, and replay/integrity evidence.
- Separate geometry from engineering evidence. Spatial geometry can synchronize dimensions, but solver results and observed pressure are not silently written into the spatial model.
- No extrapolation where the model does not authorize it. Fan curve workflows remain bounded by supplied data.
- One calculation path. The GUI is an operator shell over backend services, not a second engineering implementation.

2.3 Capability map

Domain	Stable v0.102.1	Current-main additions
Spatial design	2D/3D shared model, rooms/devices/openings, edit history, pressure visualization, sync state	IFC semantic fidelity and re-import hardening
Verification	room/project ACH, particle and pressure checks, recovery	design requirement consistency bridges
HVAC / thermal	psychrometrics, thermal loads, airflow balance, fan duty, uncertainty	additional full-precision evidence integration
Networks	duct, branch, loop, variable-friction, fan/network	room pressure/leakage network
Assurance	dossiers, consistency, provenance/replay	compliance rule packs, design assurance, snapshots, ProofGraph
Project lifecycle	strict JSON, atomic save, recovery, revisions, run history, bundles	diagnostics and additional resource hardening

2.4 Lifecycle view

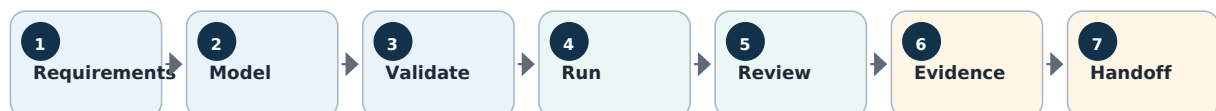


Figure 2-2. Recommended engineering lifecycle from explicit requirements through controlled evidence handoff.

A mature project does not start with the solver. It starts with explicit requirements and source data, progresses through model construction and validation, executes the selected canonical workflows, and ends with reviewed evidence that is versioned and traceable.

CHAPTER 3

Installation, Readiness, and Controlled Deployment

3. Installation, Readiness, and Controlled Deployment

3.1 Supported environment

CleanroomX supports Python 3.11, 3.12, and 3.13. The base package declares no mandatory third-party runtime dependency beyond the Python standard library. The desktop GUI requires Tk support. Native IFC ingestion is optional and requires IfcOpenShell through the `bim` extra.

3.1.1 Windows repository checkout

```
cd C:\CleanroomX
.\start-cleanroomx.ps1
```

Command Prompt / Explorer compatible launcher:

```
start-cleanroomx.cmd
```

Readiness check without opening the GUI:

```
.\start-cleanroomx.ps1 --check
```

The repository launcher prefers `.venv\Scripts\python.exe`, configures the repository src tree, and avoids ambiguity caused by a stale globally installed package.

3.1.2 Python installation

```
python -m venv .venv
python -m pip install -e .
cleanroomx-gui --check
cleanroomx-gui --demo
```

Development / validation environment:

```
python -m pip install -e .[dev]
python -m pytest -q
```

Optional native IFC support:

```
python -m pip install -e .[bim]
```

3.2 Installation acceptance procedure

Use this procedure whenever a new workstation or virtual environment is prepared for project use.

1. Record the Git commit or release tag that is being installed.
2. Create a clean Python virtual environment.
3. Install CleanroomX from the intended source or validated package.
4. Run `cleanroomx-gui --check`.
5. Launch `cleanroomx-gui --demo` and confirm the demonstration project opens.
6. Open Design 2D + 3D and confirm rooms/devices are visible.

7. Run the active demonstration analysis.
8. If the workstation is part of a controlled project, retain the installation evidence and software identity in the project record.

Installation release gate

Do not use the workstation for released project evidence if the registry readiness check fails, the intended version cannot be confirmed, or the demonstration workflow cannot execute.

3.3 Production deployment guidance

Recommended operating controls:

- run with normal user privileges;
- keep project and evidence directories under normal OS access controls;
- use organization backup/versioning for project directories;
- keep source requirements and manufacturer data separate from generated reports;
- do not place credentials, API keys, or unrelated secrets in project JSON;
- pin the CleanroomX version or commit used for controlled analyses;
- retain validation evidence appropriate to the organization's software-assurance process.

3.4 Linux / CI GUI smoke

On Debian/Ubuntu style systems:

```
sudo apt-get update
sudo apt-get install -y xvfb tk
xvfb-run -a cleanroomx-gui examples/gui_demo.cleanroomx.json --smoke
xvfb-run -a python -m pytest -q tests/test_spatial_editing_gui.py
```

This validates real Tk widget behavior under a virtual display. Headless Python tests alone do not replace GUI smoke testing for desktop deployment.

CHAPTER 4

Project Lifecycle and File Governance

4. Project Lifecycle and File Governance

4.1 Project file model

Desktop projects use `cleanroomx.project`, schema version 1. A project contains project metadata, an ordered analysis list, an optional active analysis identifier, and optional spatial layout metadata under the project block.

A project record is not only a collection of numbers. It is the anchor for engineering identity, path context, spatial state, analysis definitions, run evidence, and handoff artifacts.

Example skeleton:

```
{
  "schema": "cleanroomx.project",
  "schema_version": 1,
  "application_version": "0.102.1",
  "project": {
    "name": "Example Facility",
    "description": "Engineering screening model",
    "metadata": {}
  },
  "analyses": [
    {
      "id": "verification",
      "name": "Facility verification",
      "kind": "project_verification",
      "input": {}
    }
  ],
  "active_analysis_id": "verification"
}
```

4.2 Strict JSON boundary

CleanroomX rejects malformed JSON and non-finite constants such as NaN, Infinity, and -Infinity. Strict engineering loaders also reject duplicate object keys rather than silently using last-key-wins behavior.

Current-main project ingestion is bounded to 64 MiB. Saves use the same authority so CleanroomX does not intentionally create a normal project file it cannot reopen.

Do not repair engineering JSON with permissive tools without review

A permissive editor or script may silently normalize duplicate keys or non-standard numeric tokens. Use CleanroomX validation after any external edit.

4.3 Additive-field preservation

Within schema version 1, unknown additive strict-JSON fields at the document, project, and analysis-record levels are preserved through open/edit/save cycles. CleanroomX-owned fields remain authoritative.

This allows controlled extensions without silently deleting unknown metadata, while avoiding reinterpretation of data that the installed version does not understand.

4.4 Guarded project saves

Saved projects use atomic staging/replacement and external-write protection. CleanroomX records the stable revision opened or produced by the previous successful save. If another process changes, deletes, or replaces the project file, a normal save is blocked instead of silently overwriting the newer disk content.

Recommended response to a save conflict:

1. Do not retry by repeatedly clicking Save.
2. Use Save Project As to preserve the current in-memory work under a different path, or reopen the disk version.
3. Compare the two revisions under project change control.
4. Re-run affected analyses after any reconciliation.

4.5 Legacy migration

Supported legacy single-analysis shapes are migrated in memory. A migrated source opens as a protected unsaved copy. The first schema-v1 save must use a different destination, keeping the original legacy bytes available for comparison or rollback.

4.6 Recovery autosave

Recovery artifacts are separate from explicit project files. The GUI uses debounced/periodic recovery checkpoints and never treats a recovery artifact as the authoritative project save.

Startup recovery supports inspection, restore-as-unsaved-copy, and explicit discard. Restored recovery data must be saved to a new explicit destination before it becomes a normal saved project.

Recovery decision table

Situation	Recommended action
Source unchanged and recovery is newer	Inspect semantic differences, restore only if needed.
Source changed externally	Keep both versions; do not overwrite automatically.
Source missing	Restore as unsaved copy, then Save As.
Recovery malformed/unreadable	Preserve artifact for investigation; do not guess contents.

CHAPTER 5

Desktop GUI - Standard Operating Procedure

5. Desktop GUI - Standard Operating Procedure

5.1 Normal operator sequence

1. Create a new project or open an existing `.cleanroomx.json` project.
2. Confirm the project name, description, path context, and active analysis.
3. Add or select an analysis from the application catalog.
4. Enter or import the strict JSON input.
5. Choose Validate before execution.
6. Correct all validation errors; do not bypass the parser by editing result files.
7. Choose Run and wait for completion.
8. Review result status, diagnostics/provenance, report, and any plots.
9. Confirm the result is current for the active input.
10. Export or hand off evidence only after review.

Operator rule

Validation proves that the configured input is structurally acceptable to the selected workflow. It does not prove that the engineering assumptions are correct. Engineering review is still required.

5.2 Run ownership and Abandon

The Abandon action suppresses the pending result but does not force-terminate the Python worker thread. CleanroomX keeps the run exclusive until the worker exits so an abandoned long computation cannot overlap a new backend run.

5.3 Result freshness

Accepted completed results are bound to the canonical SHA-256 of the exact submitted analysis input. File-backed workflows also bind external dependency revisions.

A result must be treated as stale when:

- the analysis kind changes;
- the input changes after the run;
- a referenced engineering file changes or disappears;
- a synchronized geometry update changes the engineering input;
- a Save As operation changes path context and invalidates file-backed evidence.

Stale results are rejected from cache restore/export at the application boundary.

5.4 Review the four panes of evidence

For every completed analysis, review more than the summary status.

Evidence area	Review question
Input	Are units, values, names, paths, and assumptions correct?
Result JSON	What did the canonical workflow calculate and what status did it return?
Diagnostics / provenance	Is the evidence fresh, deterministic, and bound to the intended inputs?
Markdown / HTML report	Is the human-readable interpretation consistent with the JSON evidence and project boundary?

5.5 Recommended analysis naming

Use stable names that identify system and purpose, for example:

- Process Suite - Project Verification
- AHU-01 - HVAC Screening
- Supply Network - Variable Friction
- Process Pressure - Design Consistency

Avoid multiple analyses with identical display names. Stable IDs remain authoritative, but duplicate names create review ambiguity.

CHAPTER 6

Spatial Design Workspace

6. Spatial Design Workspace

6.1 Canonical 2D/3D model

The 2D editor and 3D viewer use the same spatial layout object. There is no independent 3D geometry copy. A change made to the canonical room/device model is visible in both views.

Stored spatial information includes:

Object	Key properties
Floor	stable ID, name, elevation, default ceiling height, metric units
Room	stable ID, X/Y, length, width, height, floor elevation, pressure, classification, analysis link
Device/opening	stable ID, type, room association, X/Y/Z, dimensions, orientation, wall metadata where applicable
View state	zoom, pan, 3D azimuth/elevation, snap and visibility flags
Sync baseline	analysis identity, mapping identity, last synchronized room dimensions

6.2 Supported device/opening types

The current spatial model supports doors, windows, generic wall openings, supply diffusers, return grilles, exhaust grilles, FFUs, equipment, sensors, and transfer openings.

Wall-opening records may carry width, height, wall side, orientation, and swing metadata.

6.3 Editing procedure

Room creation and editing

1. Open Design 2D + 3D.
2. Create the room and assign a unique engineering-friendly name.
3. Enter dimensions and floor elevation.
4. Set classification text only when it represents an explicit project designation.
5. Add an analysis_room_name link only when the engineering room mapping is known.
6. Use Validate after geometry changes.

Room movement and resize

- drag uses deterministic gesture-start coordinates;
- snap-to-grid may be enabled or disabled;
- a completed drag is one undo/redo transaction;
- assigned devices move with room translation;

- a no-op drag does not dirty the project or schedule autosave.

Duplication

A duplicated room receives new stable IDs and a unique name. Its assigned devices are copied with new IDs, but stored pressure, engineering room link, and synchronization baseline are deliberately not copied.

This prevents a copied geometry object from inheriting evidence or mapping identity that belongs to the original room.

6.4 3D navigation

The 3D view supports zoom, pan, reset, fit, azimuth change, elevation change, and Shift+left-drag orbit. Fit evaluates real floor and ceiling geometry rather than only resetting zoom.

6.5 Spatial validation

Spatial checks include:

- non-finite or non-positive dimensions;
- duplicate IDs and duplicate room names;
- overlapping room footprints;
- orphan or unassigned devices;
- devices outside their assigned rooms;
- invalid device elevation;
- dangling room references;
- unsupported device types and malformed view flags.

Spatial validation scope

These checks prove internal model integrity. They do not calculate airflow fields, contamination transport, constructability, fire/life-safety compliance, or regulatory cleanroom certification.

CHAPTER 7

Spatial to Engineering Synchronization

7. Spatial to Engineering Synchronization

7.1 Synchronization boundary

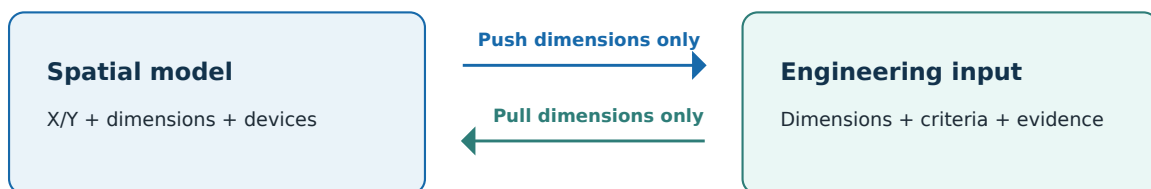


Figure 7-1. CleanroomX synchronizes shared room dimensions without silently transferring engineering evidence.

Spatial and engineering models intentionally remain separate. For supported verification workflows, push and pull operations synchronize only room length, width, and height.

- spatial X/Y placement is preserved by pulls;
- observed pressure is not copied from spatial data into engineering input;
- fresh verification pressure may be projected into the visual workspace without being persisted into geometry;
- all mappings are preflighted before mutation;
- prior results are invalidated after a real input change.

7.2 Synchronization states

State	Engineering interpretation
synchronized	Shared dimensions match the persisted synchronization baseline.
geometry newer	Baseline proves geometry changed since last synchronization.
engineering newer	Baseline proves engineering geometry changed since last synchronization.
conflicting	Both sides differ, or provenance is insufficient to decide which is authoritative.
unmapped	No valid explicit room mapping exists.

7.3 Recommended conflict procedure

1. Stop before pushing or pulling.

2. Identify the authoritative source for the current change.
3. Review room identity, name mapping, and dimensions.
4. Resolve any duplicate or missing mappings.
5. Apply one direction only.
6. Revalidate the project.
7. Re-run affected analyses.
8. Record the reason for the reconciliation in project change control if the project is controlled.

Do not use synchronization as a merge engine

Synchronization is an explicit authority transfer for shared dimensions. It is not a generic two-way merge for engineering criteria, results, pressure evidence, or project requirements.

CHAPTER 8

BIM / IFC Interoperability

8. BIM / IFC Interoperability

8.1 Purpose

The BIM/IFC bridge converts supported IFC semantics into the existing CleanroomX spatial contract and records provenance linking the spatial layout to the original IFC source and normalized semantic content.

Native .ifc ingestion requires IfcOpenShell. The base CleanroomX installation can still consume normalized semantic records supplied by another integration.

8.2 Current mapping scope

IFC entity / concept	CleanroomX behavior
IfcSpace	Room
IfcDoor	Door
IfcWindow	Window
Explicit IfcAirTerminal semantic role	Supply / return / exhaust where role is explicit
Generic IfcFlowTerminal	Equipment; no inferred HVAC airflow role
IfcSensor	Sensor
Flow controller, unitary equipment, fan, pump, furnishing element	Equipment
IfcBuildingStorey	Storey identity/elevation; single common storey may populate active floor metadata
CleanroomX_Space custom property set	Explicit Classification and AnalysisRoomName only

8.3 Geometry fidelity boundary

The current spatial contract is axis-aligned. Native import supports representable rectangular IfcSpace geometry, including 0/90/180/270 degree plan rotations. Arbitrary-angle, tilted, reflected, skewed, incomplete, or non-rectangular room geometry is rejected rather than converted to a misleading bounding box.

Space dimension provenance records whether dimensions came from explicit IFC quantities or the verified IfcOpenShell rectangular-prism fallback.

8.4 Initial import SOP

GUI route: BIM -> Import IFC Spatial Layout...

1. Select the IFC file.
2. Review the preview showing source filename, extracted room/device counts, and source SHA-256.
3. Confirm that the selected file is the intended project revision.

4. If an existing unlinked spatial layout will be replaced, review the replacement warning carefully.
5. Confirm import.
6. CleanroomX re-extracts the IFC and requires the source and semantic SHA-256 values to match the reviewed candidate before mutation.
7. Save the project explicitly after reviewing the imported layout.
8. Do not assume engineering inputs were synchronized; IFC import updates spatial state only.

CLI:

```
cleanroomx-ifc import project.cleanroomx.json facility.ifc
```

8.5 Re-import SOP

Read-only review:

```
cleanroomx-ifc plan project.cleanroomx.json facility-v2.ifc
```

Apply conflict-free re-import:

```
cleanroomx-ifc reimport project.cleanroomx.json facility-v2.ifc
```

The planner classifies unchanged, source-only, local-only, converged, added, removed, and conflicting changes. Two-sided divergent edits are blocked.

Re-import release gate

Apply only a conflict-free reviewed plan. After applying, validate the spatial model, review room identity, and then explicitly synchronize engineering dimensions if the IFC revision is intended to become authoritative for engineering geometry.

8.6 IFC resource and source-stability guards

Current main caps native IFC source files at 512 MiB before parsing and enforces the ceiling again while streaming source SHA-256. The source is hashed before and after extraction; changed or disappearing sources are rejected.

This is a provenance/resource safety control, not an engineering acceptance criterion.

CHAPTER 9

Room and Project Verification

9. Room and Project Verification

9.1 Room volume

The geometric room volume is:

$$V = \text{length} * \text{width} * \text{height}$$

Dimensions are in metres and volume is in m3.

9.2 Nominal supply ACH

$$\text{ACH} = \text{supply_airflow_m3_h} / \text{room_volume_m3}$$

ACH is a nominal supply-air change rate. It is not an airflow pattern or contaminant-distribution calculation.

9.3 Particle requirements

Room verification can compare observed particle concentration against explicit configured maximum values. CleanroomX does not generate a classification limit from the room's free-text classification field.

For controlled work, record the source of every particle criterion in the project requirements or compliance evidence.

9.4 Observed pressure and pressure cascade

Project verification can compare explicit observed room pressures using configured relationships such as:

```
{
  "higher_pressure_room": "Process",
  "lower_pressure_room": "Preparation",
  "min_delta_pa": 10
}
```

CleanroomX evaluates the configured relation from explicit evidence. Missing room pressure remains unavailable; it is not invented.

9.5 Aggregate result semantics

Status	Meaning
fail	At least one evaluated finding fails.
pass_with_unchecked	No evaluated finding fails, but unresolved/unconfigured evidence remains.
pass	Every included finding is evaluated and passing.
not_checked	Nothing in the aggregate was evaluated.

The historical passed boolean is a backward-compatible no-failure predicate. For release decisions, use status together with complete.

9.6 Worked demonstration from the packaged project

The packaged demonstration uses a Process room of 6 m x 5 m x 3 m with 2700 m³/h supply airflow.

Volume = 6 * 5 * 3 = 90 m³
ACH = 2700 / 90 = 30 1/h

The demonstration pressure relationship Process -> Preparation uses observed pressures 30 Pa and 16 Pa with a configured minimum delta of 10 Pa.

Observed delta = 30 - 16 = 14 Pa
Configured minimum = 10 Pa
Result = pass for this configured demonstration criterion

Demonstration values are not design recommendations

The values above come from the repository demo and are included only to explain software operation. They are not universal cleanroom requirements.

CHAPTER 10

Particle Decay and Recovery Qualification

10. Particle Decay and Recovery Qualification

10.1 First-order decay model

CleanroomX includes a well-mixed first-order particle-removal screening model:

$$C(t) = C0 * \exp(-(ACH/60) * \eta * t)$$

where:

- $C0$ is initial concentration;
- $C(t)$ is concentration after time t ;
- ACH is air changes per hour;
- η is effective single-pass removal fraction;
- t is time in minutes.

10.2 Recovery time

$$t = \ln(C0 / C_{target}) / ((ACH/60) * \eta)$$

If the target is greater than or equal to the initial concentration, the implemented screening function returns 0 minutes.

10.3 What the screening model does not include

The simple decay model does not resolve:

- active particle sources;
- non-uniform mixing;
- local recirculation;
- deposition;
- door-opening transients;
- leakage transport;
- room-scale velocity fields;
- CFD effects.

10.4 Measured recovery workflow

The dedicated recovery-test workflow evaluates time-series particle samples against an explicit target concentration and optional maximum recovery time. The result can retain instrument ID, sample location, occupancy state, and provenance.

Use measured recovery data for qualification evidence where required by the project. Do not replace required testing with the theoretical decay screening calculation.

CHAPTER 11

Psychrometrics, Thermal Loads, and HVAC Screening

11. Psychrometrics, Thermal Loads, and HVAC Screening

11.1 Airflow balance

For each room, steady-state net surplus is:

```
net_surplus = supply + transfer_in - return - exhaust - transfer_out
surplus_margin = net_surplus - minimum_required_surplus
```

A positive net surplus represents airflow available for exfiltration or another unmodeled outflow. A negative value means another inflow would be required to close the steady-state balance.

CleanroomX does not convert airflow surplus directly into room pressure because pressure requires an explicit leakage/opening flow-pressure model or measured relationship.

11.2 Preliminary supply-fan duty

Total static pressure:

```
P_static = P_duct + P_coil + P_terminal_filter + P_other
```

Air, shaft, and estimated electrical power:

```
Q_m3_s = airflow_m3_h / 3600
air_power_W = Q_m3_s * P_static
shaft_power_W = air_power_W / fan_efficiency
electrical_input_W = shaft_power_W / motor_efficiency
```

The workflow is preliminary steady-state screening. System effect, dirty-filter allowance, VFD/control losses, altitude correction, redundancy, acoustic acceptance, and final manufacturer selection remain external responsibilities.

11.3 Psychrometric uncertainty

For uncertain air states, CleanroomX evaluates endpoint combinations of dry-bulb temperature, relative humidity, and pressure. It reports conservative intervals for humidity ratio, enthalpy, specific volume, and dew point.

This avoids assuming monotonic behavior of every derived psychrometric quantity over the supplied uncertainty interval.

11.4 Thermal uncertainty

Cooling and heating requirement intervals are derived from the complete net-load interval and configured margin:

```
Q_cool = max(Q_net, 0) * margin_multiplier
Q_heat = max(-Q_net, 0) * margin_multiplier
```

Capacity result semantics:

Status	Capacity interpretation
pass	Available capacity covers the complete requirement interval.
fail	Complete requirement interval is above available capacity.
indeterminate	Available capacity lies inside the requirement interval.
not_checked	No available capacity is configured.

Current numerical-integrity work keeps governing airflow and capacity verdicts on full-precision calculation state, with rounding only at the reporting boundary.

CHAPTER 12

Duct, Branch, and Looped Airflow Networks

12. Duct, Branch, and Looped Airflow Networks

12.1 Duct section model

For a duct section:

```
velocity pressure = 0.5 * rho * velocity^2
friction loss = f * (L / Dh) * velocity pressure
local loss = K * velocity pressure
total section loss = friction loss + local loss
```

For a rectangular duct:

$$Dh = 2 * width * height / (width + height)$$

Friction factor may be supplied explicitly or calculated from explicit roughness and kinematic viscosity at the section airflow.

12.2 Duct-path review checklist

Before relying on a path result, confirm:

- airflow is the intended design case;
- density is appropriate for the intended screening basis;
- duct geometry and units are correct;
- friction-factor method is documented;
- local-loss coefficient includes the intended fittings only;
- path sections represent the actual critical path being reviewed.

12.3 Fixed-demand branch network

The branch solver uses mass continuity from explicit fixed terminal demands in a directed tree, then evaluates pressure loss through the duct model. It does not infer pressure-driven terminal flow or balance a looped network.

Use the loop solver when topology contains loops or parallel paths that require pressure-driven redistribution.

12.4 Looped network model

Every fixed-resistance edge uses:

$$\Delta P = R * Q * \text{abs}(Q)$$

where Q is signed airflow in m^3/s and R is $\text{Pa}/(\text{m}^3/\text{s})^2$.

A geometry-derived fixed resistance is:

$$R = 0.5 * \rho * (f * L / Dh + K) / A^2$$

Node injections must balance to zero. One reference node is assigned 0 Pa; changing the pressure reference changes the offset but not solved edge flows.

12.5 Numerical method

The loop solver validates topology, resistance, and balanced injections, constructs an initial pressure state, then solves nonlinear node continuity with damped Newton iteration. Result evidence includes node mass-balance residuals and edge pressure-law residuals.

Non-convergence raises an error rather than returning a false solved state.

CHAPTER 13

Room Pressure / Leakage Network - Current Main

13. Room Pressure / Leakage Network - Current Main

13.1 Purpose

The room pressure network is a steady-state multizone pressure solver for explicit mechanical airflows and pressure-dependent leakage/opening paths.



Figure 13-1. Simplified pressure-network concept.

At least one fixed-pressure node is required. Every unknown-pressure node must connect through configured pressure paths to a fixed-pressure boundary.

13.2 Mechanical injection

`mechanical injection = supply - return - exhaust`

The solver forms mass-balance equations around the explicit mechanical injections and pressure-driven path flows.

13.3 Power-law path

$$Q = C * \text{sign}(\Delta P) * |\Delta P|^n$$

CleanroomX accepts user-supplied exponents from 0.5 to 1.0. The coefficient and exponent are project inputs; the software does not infer them from a door/opening label.

13.4 Orifice path

$$Q = C_d * A * \text{sign}(\Delta P) * \sqrt{2 * |\Delta P| / \rho}$$

C_d , opening area, and air density are explicit inputs.

13.5 Near-zero regularization

Each path has an explicit `linearization_pressure_pa`. Near zero pressure difference, CleanroomX uses a linear continuation matched to the configured nonlinear law at the transition pressure so the Newton derivative remains usable.

13.6 Pressure targets

A target may define a high node, low node, minimum differential pressure, and optional maximum differential pressure. A physical network solve can succeed even if a target fails; the result becomes `solved_with_target_violations` rather than being collapsed into success.

13.7 Worked pressure-network demonstration

The repository demonstration uses a Process node supplying 540 m³/h to a leakage path with $C = 0.01$ m³/s/Paⁿ, $n = 1$, and a Corridor fixed at 0 Pa.

```
540 m3/h = 0.15 m3/s
Q = C * Delta P (because n = 1)
Delta P = Q / C = 0.15 / 0.01 = 15 Pa
```

This exactly illustrates the configured demo inputs. Real leakage coefficients must come from project-specific engineering data, measurements, product information, or documented assumptions.

CHAPTER 14

Fan/System Studies and Network Coupling

14. Fan/System Studies and Network Coupling

14.1 Supplied fan-curve operating point

CleanroomX accepts two or more supplied fan performance points. Airflow must be strictly increasing and pressure non-increasing. The fan curve is piecewise-linearly interpolated only between supplied points.

The system curve is:

$$\Delta P_{\text{system}} = \Delta P_{\text{fixed}} + R * Q^2$$

The operating point is the bounded intersection between the supplied fan curve and the explicit system curve.

If no intersection exists inside the supplied fan range, CleanroomX returns `no_intersection_in_supplied_range` rather than extrapolating.

14.2 Fan curve review questions

- Is the curve from the intended fan, speed, air density, and configuration?
- Are the curve points within the manufacturer's validated range?
- Does the project require correction for density, system effect, filter loading, or accessories?
- Is the operating point away from manufacturer-prohibited regions?
- Are shaft, motor, and drive limits reviewed outside CleanroomX where required?

14.3 Fan speed studies

Affinity-law transformations are applied only to the supplied reference points. The tool does not infer an acceptable VFD range, motor limit, stall/surge boundary, or manufacturer-approved operating envelope.

Reported $Q * \Delta P$ is fluid air power. It is not electrical input power unless explicit efficiency models support that calculation.

14.4 Fan-network and fan-loop integrations

CleanroomX can couple a bounded fan operating point with passive network calculations and nonlinear loop/variable-friction models. Current numerical hardening keeps the operating root and downstream network state at full precision rather than reusing rounded public display values.

This matters when an explicit project tolerance is finer than the display precision.

14.5 Pending PR #595 at manual cutoff

PR #595 proposes using the canonical full-precision standalone fan root in HVAC/fan cross-study consistency when the dossier owns the fan study. CI run #1928 passed on the PR head, but the PR was not part of the engineering baseline used for this manual.

Pending capability

Do not claim PR #595 behavior on the baseline identified in Section 1 until it is merged and the project is run on a commit that contains it.

CHAPTER 15

Variable Friction, Damper Scenarios, and Numerical Integrity

15. Variable Friction, Damper Scenarios, and Numerical Integrity

15.1 Variable-friction loop workflow

Automatic-friction edges can be iterated using solved branch flow. Each outer iteration:

1. solves the complete loop network with current edge resistances;
2. reads the full-precision solved airflow of each automatic-friction edge;
3. recomputes Reynolds number and Darcy friction;
4. derives target resistance;
5. optionally relaxes the update;
6. repeats until resistance closure is within the configured tolerance.

Directly supplied resistance and geometry-derived resistance with user-supplied Darcy factor remain fixed.

15.2 Near-zero flow handling

Reynolds-based friction is undefined at zero velocity. If an automatic-friction edge solves at or below the configured near-zero airflow threshold, its previous resistance is retained and the edge is identified as frozen near zero flow.

Current main applies this classification using full-precision edge flow, not the rounded public result.

15.3 Damper-resistance scenario studies

A damper case applies explicit resistance multipliers:

$$R_{\text{case}} = \text{multiplier} * R_{\text{base}}$$

The multiplier is not derived from actuator position or blade angle. The operator must supply resistance multipliers from an appropriate engineering basis.

Airflow redistribution metrics are calculated from canonical full-precision loop state. Rounded edge values are presentation only.

15.4 Why full precision matters

CleanroomX deliberately separates calculation state from presentation state. Rounding is appropriate for human-readable output, but it must not become a new engineering input when:

- a threshold lies near a display rounding boundary;
- two study results are compared with a fine tolerance;
- uncertainty envelopes are composed from multiple calculations;

- a verdict is reproduced from ProofGraph evidence;
- a nonlinear solver feeds another solver.

This is the core meaning of the recent numerical-integrity hardening on current main.

CHAPTER 16

Design Requirements, Consistency, Compliance, and Assurance - Current

16. Design Requirements, Consistency, Compliance, and Assurance - Current Main

16.1 Design requirements

The design-requirements workflow converts explicit room requirements and selected project/reference profiles into structured targets with provenance.

Room-level inputs may include:

- dimensions;
- minimum ACH;
- temperature/RH range;
- pressure target;
- recovery target;
- filtration requirement;
- intended process;
- occupancy and sensible-load inputs;
- contamination assumptions;
- operating mode;
- supply/return strategy.

Reference profiles are project data. They are not hard-coded regulatory standards.

16.2 Preliminary air-system design

The air-system workflow evaluates explicit drivers including minimum ACH, sensible-load airflow, outdoor airflow, exhaust/transfer balance, and minimum surplus.

Balance sizing driver:

```
max(0, exhaust + transfer_out + minimum_surplus - transfer_in)
```

The largest evaluated driver becomes the governing preliminary supply airflow. Capacity-based counts for user-supplied FFUs and terminals may be reported.

16.3 Design consistency

`design_consistency` compares only shared explicit quantities represented on both sides, including room-set identity, dimensions, minimum ACH, ACH-derived airflow, explicit sensible load, and room temperature against the configured range.

Every numeric comparison uses an explicit absolute tolerance. Missing requirement evidence remains `not_checked`; it is not promoted to `PASS`.

The workflow deliberately does not infer pressure mappings, filtration semantics, contamination control, or regulatory requirements.

16.4 Pressure design consistency

`pressure_design_consistency` compares a room's configured signed pressure target with an explicitly mapped network node-to-reference pressure difference:

```
observed_delta_pa = pressure(node) - pressure(reference_node)
```

Mappings and tolerance are explicit. Reference rooms are not inferred.

16.5 Compliance rule packs

Compliance rule packs bind explicit criteria to supplied evidence. They retain rule-pack identity/version, source/reference, criteria digest, evidence digest, expected/actual values, tolerance, and result.

Rule packs are only as complete and authoritative as the project makes them. CleanroomX does not claim that a supplied rule pack fully represents a standard.

16.6 Design assurance matrix

`design_assurance` composes:

1. one canonical design-consistency result;
2. optional pressure-design consistency;
3. one or more versioned compliance checks.

Aggregate semantics:

- `fail` if any component fails;
- `not_checked` only when every component is entirely not checked;
- `pass_with_unchecked` when no component fails but unresolved evidence remains;
- `pass` only when every component is complete and passes.

The matrix also records a deterministic traceability digest over its component evidence.

CHAPTER 17

ProofGraph Evidence Model - Current Main

17. ProofGraph Evidence Model - Current Main

17.1 Purpose

ProofGraph is a GUI-independent evidence model for requirement-to-evidence-to-verdict traceability. It is not a second engineering solver.



Figure 17-1. Requirement-to-evidence chain used by assurance and ProofGraph integrations.

17.2 Core objects

ProofGraph schema v1 defines typed records for requirements, evidence sources, evidence layers, provenance, confidence, compliance checks, findings, verdicts, corrective actions, verification runs, and the complete graph.

Evidence layers include design, calculation, simulation, commissioning, operational, and neutral declared evidence.

17.3 Fail-closed structural rules

Current main rejects, among other conditions:

- dangling references;
- finding/check requirement mismatches;
- finding evidence not declared by its check;
- verdicts referencing findings for another requirement;
- provenance dependency cycles;
- reused provenance IDs across the graph;
- conflicting explicit project IDs;
- single-finding verdict status that disagrees with the finding;- verification runs with hidden out-of-run dependencies;

- verification runs that declare checks without any outcome;
- PASS findings without supporting evidence;
- PASS findings missing required evidence kinds.

These controls prevent a structurally valid JSON object from misrepresenting the evidence relationship.

17.4 Verdict states

ProofGraph v1 uses:

- pass
- warning
- fail
- unknown
- indeterminate
- not_checked

None of unknown, indeterminate, or not_checked is equivalent to PASS.

17.5 Corrective actions

Corrective actions are modeled but must retain `requires_approval=true`. ProofGraph does not automatically apply engineering changes.

17.6 Evidence integrations

Current-main adapters include compliance rule packs, pressure design consistency, IFC design evidence, ACH design evidence, airflow-balance evidence, and thermal uncertainty capacity evidence.

The adapters preserve canonical source semantics; they do not rerun engineering through a separate implementation.

CHAPTER 18

Reporting, Bundles, Snapshots, Batch, and Diagnostics

18. Reporting, Bundles, Snapshots, Batch, and Diagnostics

18.1 Portable HTML engineering report

The desktop can export the currently selected fresh completed analysis as a self-contained HTML report. The report includes project/analysis identity, exact submitted input, normalized result, diagnostics/provenance, backend Markdown report, and an embedded machine-readable payload with deterministic SHA-256.

The report does not invoke the solver during generation. It refuses stale analysis evidence.

18.2 Portable project bundle

Portable bundles use `.cleanroomx.zip` and include:

- `manifest.json`;
- `project.cleanroomx.json`;
- deduplicated external dependencies registered by the application layer.

Example CLI:

```
cleanroomx-project-bundle export project.cleanroomx.json review.cleanroomx.zip
cleanroomx-project-bundle verify review.cleanroomx.zip
cleanroomx-project-bundle extract review.cleanroomx.zip ./review
```

Bundle verification rejects unsafe archive paths, undeclared members, integrity mismatch, unsupported schemas, encryption, unsafe compression, and configured resource-limit violations.

18.3 Design assurance snapshot

```
cleanroomx-assurance-snapshot create examples/design_assurance_demo.json design_assurance.snapshot.json
cleanroomx-assurance-snapshot verify design_assurance.snapshot.json
```

A snapshot freezes exact source bytes, normalized result, result digest, traceability digest, producing version, and whole-artifact digest. Verification replays the embedded source through the canonical assurance service.

SHA-256 proves content identity/integrity, not signer identity or source authority.

18.4 Deterministic project batch execution

```
cleanroomx-project-run project.cleanroomx.json
```

The batch runner binds execution to one exact stable project revision, executes selected analyses in persisted order, checks source revision around each attempt, and preserves per-run provenance.

Use batch execution for controlled repeatability, not as a way to bypass engineering review.

18.5 Project diagnostics

```
cleanroomx-project-check project.cleanroomx.json
cleanroomx-project-check project.cleanroomx.json --format markdown
cleanroomx-project-check project.cleanroomx.json --output diagnostics.json
```

Diagnostics checks spatial integrity, analysis input validity, engineering synchronization, run-history freshness, traceability hygiene, and analysis naming. It is read-only and does not execute engineering solvers.

Diagnostics severity

Severity	Meaning
error	Invalid/inconsistent state should be corrected before relying on the affected workflow.
warning	Actionable state requiring engineering review.
info	Incomplete but not intrinsically invalid, such as never-run analysis.

A diagnostics pass means the configured software/model/provenance checks found no error or warning. It does not mean regulatory compliance.

CHAPTER 19

Industry Review Gates and Evidence Release

19. Industry Review Gates and Evidence Release

19.1 Recommended gate model

Use these gates when CleanroomX output will support an engineering deliverable.

Gate	Minimum evidence before proceeding
G0 - Software readiness	version/commit recorded; <code>cleanroomx-gui</code> --check passed; intended environment confirmed
G1 - Project integrity	project opens cleanly; strict JSON valid; path context understood; no unresolved save conflict
G2 - Model integrity	spatial and engineering validation complete; mappings reviewed; diagnostics acceptable
G3 - Analysis execution	canonical run completed; solver/convergence status acceptable; no stale dependencies
G4 - Technical review	assumptions, units, criteria, margins, limitations, and uncertainty reviewed independently
G5 - Evidence release	fresh report/bundle/snapshot produced; software version and source revision retained; approvals recorded externally

19.2 Reviewer checklist

A reviewer should verify at least the following:

- ☐ software release/commit is identified;
- ☐ project file and relevant external dependencies are identified;
- ☐ input units and names are correct;
- ☐ criteria source is documented;
- ☐ solver status is solved/converged where required;
- ☐ no material result is stale;
- ☐ `pass_with_unchecked`, `unknown`, `indeterminate`, and `not_checked` are not presented as complete PASS;
- ☐ uncertainty bounds are explicit where used;
- ☐ rounded display values are not manually reused for fine-threshold decisions;
- ☐ BIM/IFC provenance is reviewed when spatial data came from IFC;
- ☐ report limitations match the intended downstream use;
- ☐ evidence package can be replayed or traced to exact inputs.

19.3 Recommended released package

For a controlled engineering handoff, consider retaining:

1. source project file;
2. external requirements/manufacturer/measurement files;
3. CleanroomX version or Git commit;
4. diagnostics output;
5. final analysis JSON or run bundle;
6. portable HTML/PDF report;
7. design-assurance snapshot when applicable;
8. portable project bundle when external dependencies must travel with the project;
9. review comments and approval record in the organization's document-control system;
10. relevant release/test evidence when software validation is in scope.

CHAPTER 20

Security, Data Integrity, and Operational Controls

20. Security, Data Integrity, and Operational Controls

20.1 Security boundary

CleanroomX is a local desktop/CLI application. It is not an authentication system, secret store, network service, or sandbox for hostile code.

Installed plugins are trusted executable Python packages. Plugin discovery is fail-isolated so one malformed extension does not terminate built-in startup, but plugins are still code and should be controlled like other executable software.

20.2 Project input controls

- treat project JSON and referenced engineering files as trusted project inputs;
- review paths before running file-backed analyses;
- do not store unrelated credentials in engineering files;
- use OS permissions to protect controlled records;
- back up source project files before migration or bulk edit;
- inspect portable bundles from external sources before release use.

20.3 Hashes and authenticity

CleanroomX uses SHA-256 extensively for deterministic content identity and integrity. A matching digest can show that bytes or normalized content have not changed relative to a known artifact. It does not prove:

- who authored the content;
- who approved it;
- whether a standard interpretation is correct;
- whether a source file is authoritative;
- whether a result is suitable for certification.

Authentication and approval belong to the organization's document-control and identity systems.

CHAPTER 21

Troubleshooting and Failure Handling

21. Troubleshooting and Failure Handling

21.1 GUI does not start

Check in this order:

1. run `cleanroomx-gui --check`;
2. verify Python version is 3.11-3.13;
3. verify Tk support exists;
4. on Windows, use the repository launcher to avoid PATH ambiguity;
5. confirm the intended CleanroomX build is imported rather than an older installed package;
6. inspect the exception before modifying project files.

21.2 Project will not open

Possible causes include malformed JSON, non-finite constants, invalid UTF-8, unsupported schema/version, duplicate IDs, invalid active-analysis reference, oversized project file, or unsupported analysis kind.

Do not hand-edit around a schema error without keeping the original source. Make a copy, correct one issue at a time, and revalidate.

21.3 Save blocked because the file changed externally

This is a protection, not a defect.

- Save As to a different file to preserve in-memory work; or
- reopen the disk version and reconcile under change control.

Do not attempt to bypass the external-write guard by creating same-file aliases.

21.4 Result disappeared after editing input

Expected behavior. Result freshness is bound to the exact submitted input identity. Revalidate and rerun.

21.5 IFC re-import reports conflict

Review the per-entity plan. A conflict means both the local spatial object and the IFC-derived source changed differently relative to the prior baseline. Decide which source is authoritative and reconcile intentionally. Conflict-free source-only changes can be applied while preserving stable CleanroomX IDs.

21.6 Network solver does not converge

Review:

- connectivity and reference boundary;

- sign convention and balanced injections;
- resistance/leakage parameter magnitude;
- near-zero linearization setting;
- unrealistic or non-finite input scales;
- configured iteration/tolerance values.

Do not report a manually edited solved state. Correct the model and rerun.

21.7 Fan study reports no intersection

`no_intersection_in_supplied_range` means the configured fan and system curves did not cross inside the supplied fan data. CleanroomX deliberately does not extrapolate. Review the supplied fan data, system model, or equipment selection basis.

21.8 PASS but incomplete

`pass_with_unchecked` means evaluated evidence passed but unresolved evidence remains. Review every unchecked item before treating the project as complete.

CHAPTER 22

Validation, Change Control, and Rollback

22. Validation, Change Control, and Rollback

22.1 Stable release evidence

The v0.102.1 final spatial production baseline recorded:

- 1008 passing tests on Python 3.11;
- 1008 passing tests on Python 3.12;
- 1008 passing tests on Python 3.13;
- Windows PowerShell and CMD launcher smoke;
- clean wheel build/install checks;
- Python 3.13 installed Tk/Xvfb GUI smoke;
- Release 2 compatibility/provenance gates;
- deterministic spatial and autosave race regressions.

The authoritative detailed records remain `VALIDATION.txt`, `TEST_EVIDENCE.md`, and `CHANGELOG.md` in the repository.

22.2 Controlled change procedure

For project use after a software update:

1. identify the old and new CleanroomX versions/commits;
2. review changelog and affected workflows;
3. run the organization's required software verification;
4. reopen representative projects;
5. rerun affected analyses rather than reusing generated results from an old build;
6. compare material outputs and explain differences;
7. approve the new build for controlled use before release work.

22.3 Code rollback

Use normal Git revert and CI on shared repository history. Do not rewrite shared history unless repository policy explicitly allows it.

22.4 Project rollback

Restore a preserved earlier project revision or original legacy source. CleanroomX does not synthesize reverse migration to historical formats.

Generated result files should be regenerated from preserved inputs under the selected validated software version rather than manually edited to match an earlier report.

CHAPTER 23

Command-Line Quick Reference

23. Command-Line Quick Reference

The installed entry points on the current project package include:

Command	Primary use
cleanroomx	Core room calculations / verification helpers
cleanroomx-gui	Desktop application, readiness check, demo/smoke
cleanroomx-project-run	Deterministic saved-project batch execution
cleanroomx-project-bundle	Export / verify / extract portable bundles
cleanroomx-project-check	Read-only project diagnostics
cleanroomx-ifc	IFC import / plan / reimport
cleanroomx-hvac	HVAC project screening
cleanroomx-recovery-test	Measured recovery qualification
cleanroomx-uncertainty	Room uncertainty
cleanroomx-qualification	Qualification workflow
cleanroomx-duct-flow	Duct path pressure-loss analysis
cleanroomx-loop-flow	Fixed-resistance loop network
cleanroomx-pressure-network	Room pressure/leakage network
cleanroomx-loop-friction	Variable-friction loop
cleanroomx-thermal-uncertainty	Thermal/HVAC bounded uncertainty
cleanroomx-psychrometric-uncertainty	Psychrometric bounded uncertainty
cleanroomx-fan-curve	Fan/system operating point
cleanroomx-fan-uncertainty	Fan uncertainty
cleanroomx-fan-speed	Fan speed / affinity study
cleanroomx-fan-duct	Fan + duct network composition
cleanroomx-fan-network	Fan + passive network
cleanroomx-fan-loop	Fan + loop network
cleanroomx-fan-loop-friction	Fan + variable-friction loop
cleanroomx-fan-loop-friction-speed	Speed study on fan/variable-friction loop
cleanroomx-fan-loop-friction-uncertainty	Uncertainty on fan/variable-friction loop
cleanroomx-fan-loop-uncertainty	Fan/loop uncertainty

Command	Primary use
cleanroomx-fan-loop-speed	Fan/loop speed study
cleanroomx-damper-study	Loop damper resistance scenarios
cleanroomx-dossier	Engineering dossier composition
cleanroomx-consistency	Cross-study consistency
cleanroomx-assurance-snapshot	Create / verify deterministic assurance snapshots

Use --help on the installed command to confirm exact options in the software version being used.

CHAPTER 24

Engineering Formula Summary

24. Engineering Formula Summary

Calculation	Implemented relationship / interpretation		
Room volume	$V = L * W * H$		
Nominal ACH	$ACH = \text{supply_m3_h} / V_{m3}$		
Particle decay	$C(t) = C_0 \exp(-(ACH/60) \text{ eta } t)$		
Recovery time	$t = \ln(C_0/C_{\text{target}}) / ((ACH/60) \text{ eta})$		
Airflow surplus	$\text{supply} + \text{transfer_in} - \text{return} - \text{exhaust} - \text{transfer_out}$		
Duct velocity pressure	$0.5 \text{ rho } v^2$		
Darcy friction loss	$f (L/Dh) (0.5 \text{ rho } v^2)$		
Local duct loss	$K (0.5 \text{ rho } v^2)$		
Rectangular hydraulic diameter	$2WH/(W+H)$		
Supply fan fluid power	$Q * \Delta P$		
Shaft power	$\text{fluid power} / \text{fan efficiency}$		
Electrical input estimate	$\text{shaft power} / \text{motor efficiency}$		
Loop edge pressure law	$\Delta P = R Q \text{ abs}(Q)$		
Geometry-derived loop R	$0.5 \text{ rho } (fL/Dh + K) / A^2$		
Pressure path power law	$\dot{Q} = C \text{ sign}(\Delta P)$	ΔP	n
Pressure path orifice	$\dot{Q} = C_d A \text{ sign}(\Delta P) \sqrt{2}$	ΔP	$/\text{rho}$
Fan system curve	$\Delta P_{\text{system}} = \Delta P_{\text{fixed}} + R Q^2$		

All formula use remains subject to the workflow-specific assumptions and boundaries described in the relevant chapter.

CHAPTER 25

External Standards and Reference Map

25. External Standards and Reference Map

CleanroomX does not embed these documents as automatic acceptance criteria. They are listed as authoritative context that project engineers may use to define requirements, test plans, or rule packs.

Reference	Current verified official description / relevance
ISO 14644-1:2015	Classification of air cleanliness by particle concentration. ISO reports Edition 2 (2015) as current after review/confirmation. Official: https://www.iso.org/standard/53394.html
ISO 14644-2:2015	Monitoring to provide evidence of cleanroom performance related to air cleanliness by particle concentration. Official: https://www.iso.org/standard/53393.html
ISO 14644-3:2019	Test methods for cleanrooms and clean zones. Official ISO publication page.
EU GMP Annex 1 (2022 revision)	Manufacture of Sterile Medicinal Products; European Commission publication dated 25 Aug 2022, with entry into operation dates stated by the Commission. Official: https://health.ec.europa.eu/latest-updates/revision-manufacture-sterile-medicinal-products-2022-08-25_en
ASHRAE Handbook - HVAC Applications, Clean Spaces	Clean-space design context including contamination control, airflow patterns, pressurization, testing/commissioning concepts. Official handbook: https://handbook.ashrae.org/
ASHRAE Design Guide for Cleanrooms	Practical cleanroom fundamentals, environmental control systems, testing, commissioning, qualification, and industry applications. Official: https://www.ashrae.org/technical-resources/bookstore/ashrae-design-guide-for-cleanrooms
NIST CONTAM 3.4 documentation	Multizone airflow and contaminant transport reference supporting pressure/airflow model forms. Official: https://www.nist.gov/publications/contam-user-guide-and-program-documentation-version-34
buildingSMART IFC 4.3.2.0	Official IFC 4.3 release documentation for BIM data exchange; buildingSMART identifies 4.3.2.0 as the latest official IFC release and ISO 16739-1:2024 publication. Official: https://standards.buildingsmart.org/IFC/RELEASE/IFC4_3/

Standards use

The standard or regulation applicable to a real facility depends on jurisdiction, industry, process, product, project contract, and lifecycle stage. Use licensed/current copies and qualified engineering/regulatory interpretation. A CleanroomX rule pack or project profile is project data, not a substitute for the source document.

CHAPTER 26

Final Release Checklist

26. Final Release Checklist

Before releasing a CleanroomX-based engineering package, complete the following checklist or an organization-approved equivalent.

Software and project identity

- ☐ software version / Git commit recorded;
- ☐ project file identified and protected from uncontrolled overwrite;
- ☐ external engineering dependencies identified;
- ☐ project diagnostics reviewed;
- ☐ recovery or alternate revisions reconciled.

Model and input review

- ☐ room/device identity reviewed;
- ☐ engineering geometry synchronized intentionally;
- ☐ units verified;
- ☐ criteria source documented;
- ☐ manufacturer data revision documented where used;
- ☐ uncertainty basis documented where used;
- ☐ IFC source/revision documented where used.

Analysis review

- ☐ validation passed;
- ☐ solver converged / valid workflow status obtained;
- ☐ result is fresh for the current input;
- ☐ external dependency fingerprints are current;
- ☐ residuals / margins / limits reviewed;
- ☐ incomplete statuses are not represented as complete PASS;
- ☐ numerical precision is not degraded by manual reuse of rounded values.

Evidence and handoff

- ☐ final report generated from fresh evidence;
- ☐ project bundle created when dependencies must be transferred;

- ☐ assurance snapshot created where applicable;
- ☐ released files stored under controlled access;
- ☐ independent technical review completed;
- ☐ approval recorded outside CleanroomX in the organization's approval system.

CHAPTER 27

Glossary and Status Semantics

27. Glossary and Status Semantics

Term	Meaning in CleanroomX
Canonical input identity	Deterministic SHA-256 of the exact normalized/submitted input used by the application execution path.
Fresh result	Result whose analysis kind/input and required external dependency revisions still match the completed run evidence.
Strict JSON	JSON boundary that rejects malformed syntax, duplicate keys where enforced, and non-finite numeric constants.
Spatial layout	Canonical room/device geometry stored in project metadata and shared by 2D/3D views.
Sync baseline	Persisted shared-dimension/mapping evidence used to determine whether geometry or engineering changed since the last synchronization.
Pass with unchecked	No evaluated failure, but unresolved criteria remain; not a complete PASS.
Unknown	Evidence/provenance is insufficient for a verified verdict.
Indeterminate	Canonical analysis was evaluated but its result interval overlaps the decision boundary.
Not checked	Known criterion/check was not evaluated.
ProofGraph	Traceability/evidence graph; not a separate engineering solver.
Content digest	Deterministic SHA-256 identity used for integrity/provenance, not a digital signature.
Portable bundle	Integrity-checked ZIP containing project and registered external dependencies.
Design assurance snapshot	Self-contained artifact binding source bytes, normalized result, traceability, producing version, and replay evidence.

CHAPTER 28

Current Development Status at Manual Cutoff

28. Current Development Status at Manual Cutoff

This appendix isolates development status from the operating procedures so normal users do not have to interpret repository chronology while using the software.

28.1 Stable v0.102.1

Stable release scope includes the Release 2 project lifecycle and final synchronized spatial production closure:

- 2D/3D shared spatial model;
- dimension-only bidirectional synchronization;
- deterministic drag/edit history;
- windows and generic openings;
- viewport fit/reset/orbit controls;
- atomic persistence and guarded saves;
- recovery, revisions, run history, bundles, reports;
- project batch execution;
- strict engineering JSON;
- precision-safe core HVAC composition;
- explicit verification completeness;
- v0.91-v0.95 compatibility/provenance gates.

28.2 Current-main engineering additions through PR #594

The engineering baseline used for this manual includes current-main work such as:

- room pressure/leakage network;
- design requirements and preliminary air-system design;
- design consistency and pressure design consistency;
- compliance rule-pack and design assurance composition;
- design assurance snapshots;
- project diagnostics;
- ProofGraph foundation and multiple evidence adapters;
- conflict-aware IFC re-import and source-stability/resource hardening;
- full-precision pressure-design, fan/network, loop, thermal, cross-study, uncertainty, and damper composition hardening;

- additional ProofGraph structural-integrity rules.

28.3 Pending PR #595

At the manual cutoff, PR #595 was open and its exact head e2fd382181b512d8ec786747db2bdc3954edb968 had CI run #1928 with conclusion SUCCESS. Its proposed behavior is not part of the engineering baseline identified on the cover.

28.4 Authoritative repository records

For release engineering and code-history audit, use:

- README.md
- ARCHITECTURE.md
- CHANGELOG.md
- VALIDATION.txt
- TEST_EVIDENCE.md
- workflow-specific documents under docs/

This manual is the operator/engineering guide; the repository records remain authoritative for exact commit-level development history.